

Advanced Algorithms

Assignment 3 – Group Programming Task

Chosen Problem: **Bin Packing**

Prepared by:

Byron Lester 14386344

Wei Ming Edward Ong 14005817

Table of Contents

1	The Problem	3
2	Our Implementation – Assumptions, Modifications & Restrictions	3
2.1	<i>Assumptions.....</i>	3
2.2	<i>Modifications and Restrictions</i>	4
2.2.1	Sorting Items in Decreasing Order (Modification)	4
2.2.2	Fixed Bin Capacity (Restriction)	4
2.2.3	Rejection of Oversized Items (Restriction)	4
2.2.4	Absence of Bin Rebalancing (Restriction)	4
3	Outline of Algorithms	5
4	Theoretical Complexity	5
4.1	<i>Time Complexity</i>	5
4.2	<i>Space Complexity</i>	6
5	Complexity of Our Implementation.....	6
6	Tests	6
7	Insights	8
8	Conclusion.....	9
9	References	10

1 The Problem

The bin packing problem is a combinatorial optimisation problem where the objective is to pack a set of items of varying sizes into a minimum number of containers or ‘bins’, without exceeding each bin’s fixed capacity (Gupta, 2023; Wikipedia contributors, 2024a). This problem has many real-world applications, including loading trucks, packing containers, splitting a network prefix into subnets, and creating file backups in media (Wikipedia contributors, 2024a).

The problem is notable in computing science due to its utility in real world scenarios and for being computationally NP-hard. Despite its worst-case hardness, achieving optimal solutions (solutions that return the precise minimum number of bins) is possible with advanced algorithms (Korf, 2002; Wikipedia contributors, 2024a). However, in practice, these algorithms are often slow and challenging to implement due to their complexity and poor scalability performance, particularly with significantly large problem instances (Korf, 2002).

Therefore, various heuristic algorithms considered ‘approximately-optimal’ have been developed that more effectively balance accuracy, performance, and scalability (Gupta, 2023; Korf, 2002; Wikipedia contributors, 2024a). Examples of such algorithms include First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) (Gupta, 2023; Wikipedia contributors, 2024a).

2 Our Implementation – Assumptions, Modifications & Restrictions

2.1 Assumptions

1. All items have positive weights.
2. If an item’s weight exceeds the pre-defined capacity of the bins, -1 will be returned to indicate the problem instance is unsolvable.
3. The item list is known before the algorithm is run, making it suitable for offline algorithms – algorithms with complete knowledge of the items and their weights in advance - such as FFD and BFD.
4. The capacity of the bins is fixed, remaining constant throughout the solution.
5. The number of bins necessary to accommodate the items is to be returned.

2.2 Modifications and Restrictions

2.2.1 Sorting Items in Decreasing Order (Modification)

- **Description:** In this implementation, we have chosen to do the 'offline' format of the problem, where the algorithm has knowledge of all items and their weights before the algorithm starts. Adopting this format of the problem is a deliberate modification as it allows for improved approximation performance.
- **Impact:** This sorting step is not part of the 'online' 'First Fit' and 'Best Fit' approaches but is a modification that allows for improved approximation performance. This sorting step uses 'std::sort' from the C++ standard library, which is known to be of $O(n \log n)$ time complexity.

2.2.2 Fixed Bin Capacity (Restriction)

- **Description:** Both algorithms assume that all bins have a fixed capacity. This is a common restriction of standard bin-packing problems.
- **Impact:** Fixing bin capacity allows for an easier implementation and predictable behaviour. Although this restricts flexibility, variable bins would require a significantly more complex solution.

2.2.3 Rejection of Oversized Items (Restriction)

- **Description:** The algorithm checks if any item exceeds the capacity of the bins and returns -1 if one does.
- **Impact:** This restriction simplifies the problem by excluding scenarios where oversized items may need to be split across multiple bins. This is common in standard bin packing problems.

2.2.4 Absence of Bin Rebalancing (Restriction)

- **Description:** Once an item is placed in a bin, the algorithms do not attempt to rebalance the bins, even if a better configuration may emerge as more items are processed.

- **Impact:** This restriction improves algorithm efficiency and simplicity at the cost of solution accuracy.

3 Outline of Algorithms

To solve the bin-packing problem, the first fit decreasing and best fit decreasing heuristic algorithms have been implemented due their simplicity, efficiency and accuracy in approximating.

1. **First Fit Decreasing (FFD):** First fit decreasing works by first sorting the input of items in decreasing order and then applying the first fit algorithm. This algorithm involves looping through each item and either placing it in the first bin that can accommodate it or, if none exists, opening a new empty bin and placing the item in it. This process repeats until all items are placed. The worst-case approximation ratio of first fit decreasing is $11/9 \cdot \text{OPT}(I) + 6/9$ where $\text{OPT}(I)$ is the optimal number of bins (Wikipedia contributors, 2024a, 2024b). This approximation ratio is tighter (closer to the optimal number of bins) when compared to the non-sorted variant of the algorithm ‘first fit’ (Wikipedia contributors, 2024b).
2. **Best Fit Decreasing (BFD):** Similarly, best fit decreasing first sorts the input of items in decreasing order and then applies the best fit algorithm (Jones, n.d.; Wikipedia contributors, 2023). When an item arrives for processing, it looks for the bin the with least remaining space but with enough to fit the item. If one exists, the item is placed in the bin and if not, a new bin is opened at the item is placed there. This process repeats until all items are placed. The approximation ratio of BFD is identical to FFD (Wikipedia contributors, 2024b).

4 Theoretical Complexity

4.1 Time Complexity

Both FFD and BFD require a sorting step using the standard library sort function. This function is known to be $O(n \log n)$. After sorting, we check for each item if a bin exists that meets the criteria of either FFD or BFD. In the worst case, the algorithm may need

to check all bins before finding a fit. If there are n items and up to n bins, in the worst case, this results in a time complexity of $O(n^2)$.

4.2 Space Complexity

For both FFD and BFD, in the worst case, use an array of bins that can scale with the number of items. Thus, the space complexity is $O(n)$.

5 Complexity of Our Implementation

The complexity of our implementation is faithful to typical FFD and BFD implementations for the offline bin packing problem. Therefore, the time complexity and space complexity can be described by the theoretical complexity mentioned above, which is $O(n^2)$ and $O(n)$ respectively.

6 Tests

To verify the correctness of the First Fit Decreasing and Best Fit Decreasing algorithms, a series of tests have been implemented via the GoogleTest framework. These tests include:

- **SimpleTest:** This test takes a simple set of items ($n=6$) and attempts to place them in bins with max capacity of 10. The optimal number of bins to be returned is two.
- **ThreeBins:** This test demonstrates the algorithms' ability to compact a set of items ($n=6$) into an optimum number of three bins.
- **FourBins:** This test demonstrates the algorithms' ability to compact a set of items ($n=7$) into an optimum number of four bins.
- **EmptyItems:** This test verifies that algorithms' ability to handle the edge case in which no items are provided. As there are no bins to pack, this test expects zero to be returned.
- **SingleItem:** This test verifies the algorithms' ability to assign one item to a bin.

- **SingleItemWithExtraSpace:** This test verifies the algorithms' ability to accommodate multiple items in one bin and still have remaining space after it's been packed.
- **SingleBinWithNoSpace:** This test verifies the algorithms' ability to recognise when a bin does not have the room to accommodate multiple items, resulting in a new bin being generated.
- **SingleBinWithJustEnoughSpace:** This test verifies the algorithms' ability pack an item with weight that matches that of the maximum bin capacity.
- **SingleBinFitMultiple:** This test verifies the algorithms can assign multiple items to a bin and fill it completely.
- **SingleItemWithNoSpace:** This test verifies the algorithms' return the invalid value (-1) when an item weight exceeds the maximum capacity of the bins
- **ManySmallItems:** This test verifies the algorithms' efficiently pack a moderately large array of 1000 items, each of size 1, into bins of capacity equal to 10. As each bin can hold exactly 10 items, it can be easily known that the optimal number of bins to accommodate the items is 100.
- **LargeNumberOfItems:** This test verifies the algorithms' ability to provide a positive result with a large input of items (n=10000) each with a weight of 2. This test expects the optimal number of bins to be returned to be 2000. However, the primary purpose of this test is to highlight the algorithms' ability to return a result when an intensive input is given.

This suite of tests effectively verifies the correctness of the algorithm by covering a range of typical and edge case scenarios to ensure complete functionality of the algorithms. The *SimpleTest*, *ThreeBins*, and *FourBins* tests validate the algorithms' ability to take a small set of items and pack them optimally into bins, highlighting accurate and efficient bin utilisation with small inputs. Edge cases area also addressed: *EmptyItems* confirms that zero is returned when no items are provided; *SingleItemWithNoSpace* checks that an error value is returned when an item exceeds the bin capacity, while *ManySmallItems* and *LargeNumberOfItems* evaluate the algorithms' capability in processing larger inputs and efficiently packing thousands of items into an expected number of bins.

In summary, this test suite validates the functionality, robustness, and efficiency of the algorithms.

7 Insights

The process of solving the bin-packing problem and implementing the First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) algorithms stimulated the following insights:

- 1. Efficiency vs Optimality:** In the process of discovering solutions, it became clear there is an important trade-off between efficiency and optimality in the bin-packing problem. While First Fit Decreasing and Best Fit Decreasing are relatively efficient, they are only capable of providing approximately optimal solutions, unable to return optimal results under every circumstance. This insight highlights the unavoidable trade-off between achieving optimality and computational time, particularly in NP-hard problems such as bin packing.
- 2. The importance of the heuristic:** The use of sorting (decreasing order), utilised in FFD and BFD, significantly improves solution quality compared to their unsorted variants (First Fit, Best Fit). This insight highlights how a simple modification in the heuristic can provide substantial improvements to performance. However, this optimisation is dependent on the format of the problem, which will be elaborated on in the following point.
- 3. Format of the problem:** The bin-packing problem can come in various formats. For example, the format of the problem the FFD and BFD algorithms are applicable to the 'offline' format of the problem. This version allows knowledge of all items and their sizes to be known upfront, allowing for preprocessing steps such as sorting the items in decreasing order. However, in the 'online' format of the problem, items arrive sequentially, requiring the algorithm to place the item in the bin without knowledge of future items. As this format does not allow for preprocessing, it is more challenging to optimise. This insight demonstrates the importance of identifying the appropriate format of the bin packing problem, as it has significant effects on algorithm behaviour, efficiency, and the optimality of the solution.
- 4. Practicality of Approximation Algorithms:** Learning about the bin-packing problem demonstrated the practicality of approximation algorithms like FFD and BFD. Although these algorithms are not perfect, they strike a practical balance for real-world use cases where exact solutions are too computationally expensive. This insight highlights the value of approximation in situations where the problem is challenging to solve both optimally and efficiently, and when time and resources are limited.

8 Conclusion

To conclude, the bin packing problem is a complex yet practical challenge in optimisation, notable for its NP-hard complexity and wide applicability in real-world scenarios. Through implementation and analysis of the First Fit Decreasing (FFD) and Best Fit Decreasing (BFD algorithms, it has been demonstrated that approximately optimal solutions can provide a valuable balance in accuracy and efficiency, meeting the practical demands of the bin packing problem.

9 References

Gupta, D. (2023, April 5). *Bin Packing Problem (Minimize number of used Bins)*.

GeeksforGeeks. <https://www.geeksforgeeks.org/bin-packing-problem-minimize-number-of-used-bins/>

Jones, E. (n.d.). *Bin-Packing Algorithms*. Eisah Jones. Retrieved November 2, 2024, from

<https://www.eisahjones.com/sorting-algorithms>

Korf, R. E. (2002). *A New Algorithm for Optimal Bin Packing*.

Wikipedia contributors. (2023). Best-fit bin packing. In *Wikipedia*.

https://en.wikipedia.org/w/index.php?title=Best-fit_bin_packing&oldid=1190559703

Wikipedia contributors. (2024a). Bin packing problem. In *Wikipedia*.

https://en.wikipedia.org/w/index.php?title=Bin_packing_problem&oldid=1237237290

Wikipedia contributors. (2024b). First-fit-decreasing bin packing. In *Wikipedia*.

https://en.wikipedia.org/w/index.php?title=First-fit-decreasing_bin_packing&oldid=1210055959